

Enterprise Candy Planning API Documentation

The Enterprise Candy Planning API is an API used for getting, updating, deleting, or adding entries to the Enterprise Candy Planning database and the items, inventory, distributors, and distributor prices. Below is a list of various endpoints that can be used, as well as how to use them and what to expect in return.

Get Requests:

`/items`

Returns a Json object with all items in the inventory system. This includes information such as the name of the item, the item id, the stock of the item, and the total capacity of the item.

`/items/outofstock`

Returns a Json object with all items in the inventory system that are out of stock(stock is 0). This includes information such as the name of the item, the item id, the stock of the item, and the total capacity of the item.

`/items/overstock`

Returns a Json object with all items in the inventory system that are overstocked(stock is greater than capacity). This includes information such as the name of the item, the item id, the stock of the item, and the total capacity of the item.

`/items/lowstock`

Returns a Json object with all items in the inventory system that are low on stock(stock is less than 35% of capacity). This includes information such as the name of the item, the item id, the stock of the item, and the total capacity of the item.

`/items/:id`

Example: `/items/3`

Return a Json object of an item given the id by the user. This includes information such as the name of the item, the item id, the stock of the item, and the total capacity of the item. If no item matches the id, a blank Json object is returned.

`/items/cheapest/:id`

Example: `/items/cheapest/10`

Returns a Json object with a list of the cheapest price to completely stock an item for each distributor given the item id by the user. This includes information such as the cost to completely restock the item from the distributor and the distributor name. If no item matches the id or no distributors carry this item, a blank Json object is returned.

/distributors

Returns a Json object with all distributors in the system. This includes information such as the name of the distributor and the id of the distributor.

/distributors/:id

Example: /distributors/3

Returns a Json object with all items a given distributor carries given the distributor id from the user. This includes information such as the price of the item, the name of the item, and the item id. If no distributor matches the id or the distributor doesn't carry any items, a blank Json object is returned.

/distributors/item/:id

Example: /distributors/item/2

Returns a Json object with all distributors that carry a given item given the item id. This includes information such as the cost of the item, the name of the distributor, and the id of the distributor. If no item matches the id or no distributors carry this item, a blank Json object is returned.

/version

Returns the string "TopBloc Code Challenge v1.0"

Post Requests:

/items/add

Example: /items/add?name=Watchamacallit

Input requirements: name(Unique, not blank string)

Creates an item to add to the database given the name by the user. The name of the item must be unique and not blank in order for the item to be correctly added. Returns a Json object with a message that this was successful, or an error message if unsuccessful.

/items/addtoinventory

Example: /items/addtoinventory?itemId=140&stock=10&capacity=12

Input requirements: itemId(not blank integer), stock(not blank integer), capacity(not blank integer)

Creates an inventory entry for an item given the itemId, stock, and capacity by the user. The itemId must reference an item in the database, and stock and capacity must be an integer for the entry to be correctly added. Returns a Json object with a message that this was successful, or an error message if unsuccessful.

/distributors/add

Example: /distributors/add?name=Willy Wonka

Input requirements: name(Unique, not blank string)

Creates a distributor to add to the database given the name by the user. The name of the distributor must be unique and not blank in order for the distributor to be correctly added. Returns a Json object with a message that this was successful, or an error message if unsuccessful.

/distributors/prices/add

Example: /distributors/prices/add?distributor=4&item=8&cost=3

Input requirements: distributor(not blank integer), item(not blank integer), cost(not blank float)

Creates a distributor price entry for an item and distributor given the distributor id, item id, and cost by the user. The distributor id must reference a distributor in the database, the item id must represent an item in the database, and the cost must be a not blank float for the entry to be correctly added. Returns a Json object with a message that this was successful, or an error message if unsuccessful.

Put Requests:

/items/update/:id

Example: /items/update/2?stock=2&capacity=20

Input requirements: stock(not blank integer), capacity(not blank integer)

Updates an item's stock and/or capacity in inventory given the item id by the user.

Returns a Json object with a message that this was successful, or an error message if unsuccessful.

/distributors/prices/update/:id

Example: /distributors/prices/update/2?item=4&cost=0.45

Input requirements: item(not blank integer), cost(not blank float)

Updates a distributor's price for an item given the distributor id, item id, and the cost by user. Returns a Json object with a message that this was successful, or an error message if unsuccessful.

Delete Requests:

/items/delete/:id

Example: /items/delete/2

Deletes an item from the database, including the item, the items inventory, and an item's price of each of its distributors given the item id by the user. Returns a Json object with a message that this was successful, or an error message if unsuccessful.

/distributors/delete/:id

Example: /distributors/delete/2

Deletes a distributor from the database, including the distributor and any of the distributors pricing given the distributor id by the user. Returns a Json object with a message that this was successful, or an error message if unsuccessful.